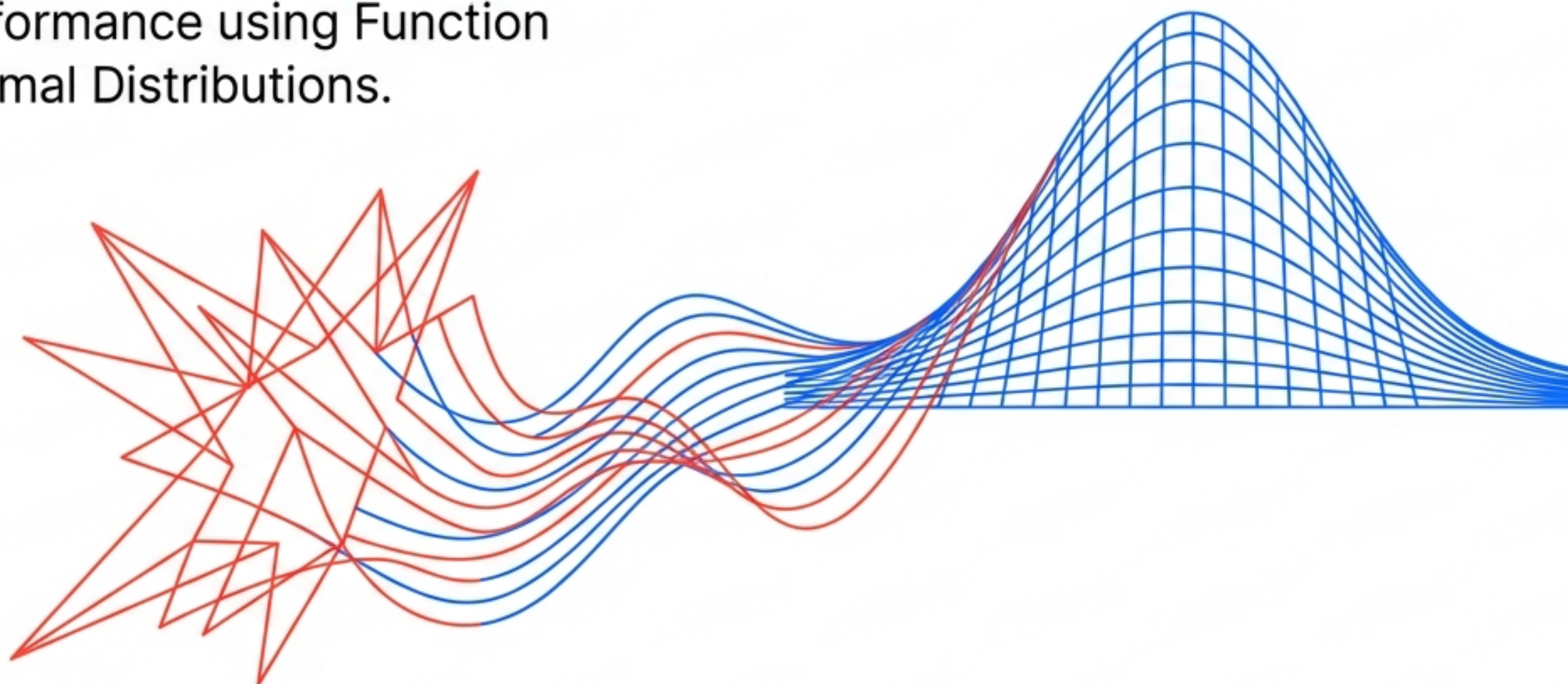


# Mathematical Transforms in Feature Engineering

Series: Feature Engineering / Chapter 4

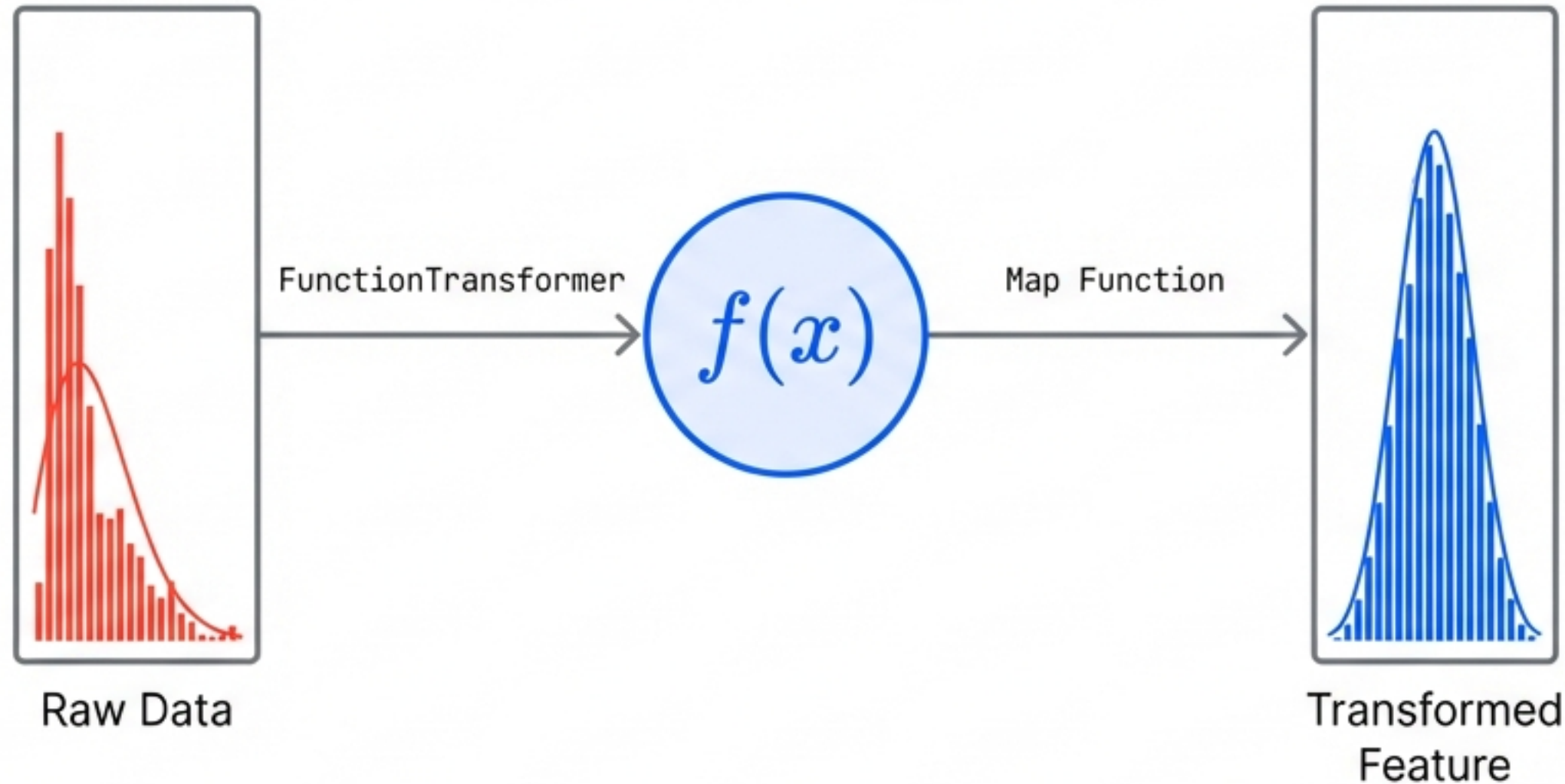
Optimising Model Performance using Function Transformers and Normal Distributions.



Concept: Transformation. Goal: Normality. Outcome: Accuracy.

# Reshaping the Probability Distribution

Feature Engineering extends beyond encoding and imputation. It involves altering the fundamental geometry of data to assist algorithmic learning.



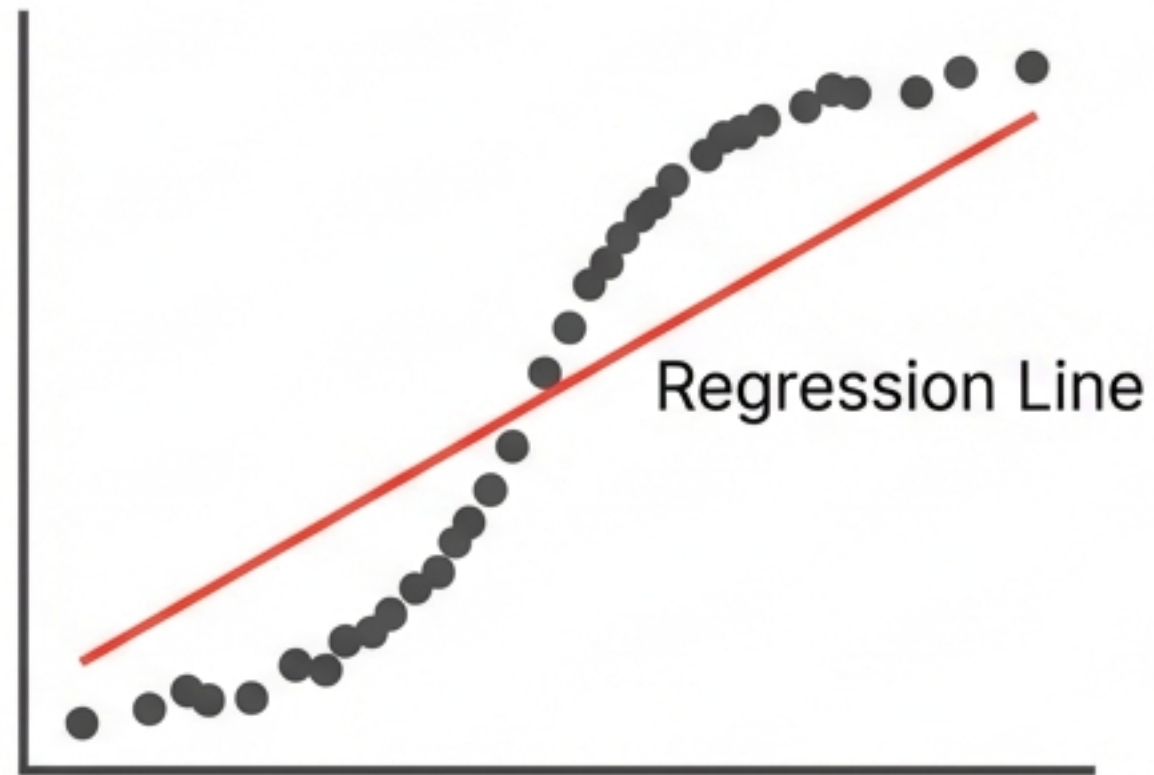
## The Toolbox

- Log Transform (Primary Focus)
- Reciprocal Transform
- Power Transforms (Square & Square Root)



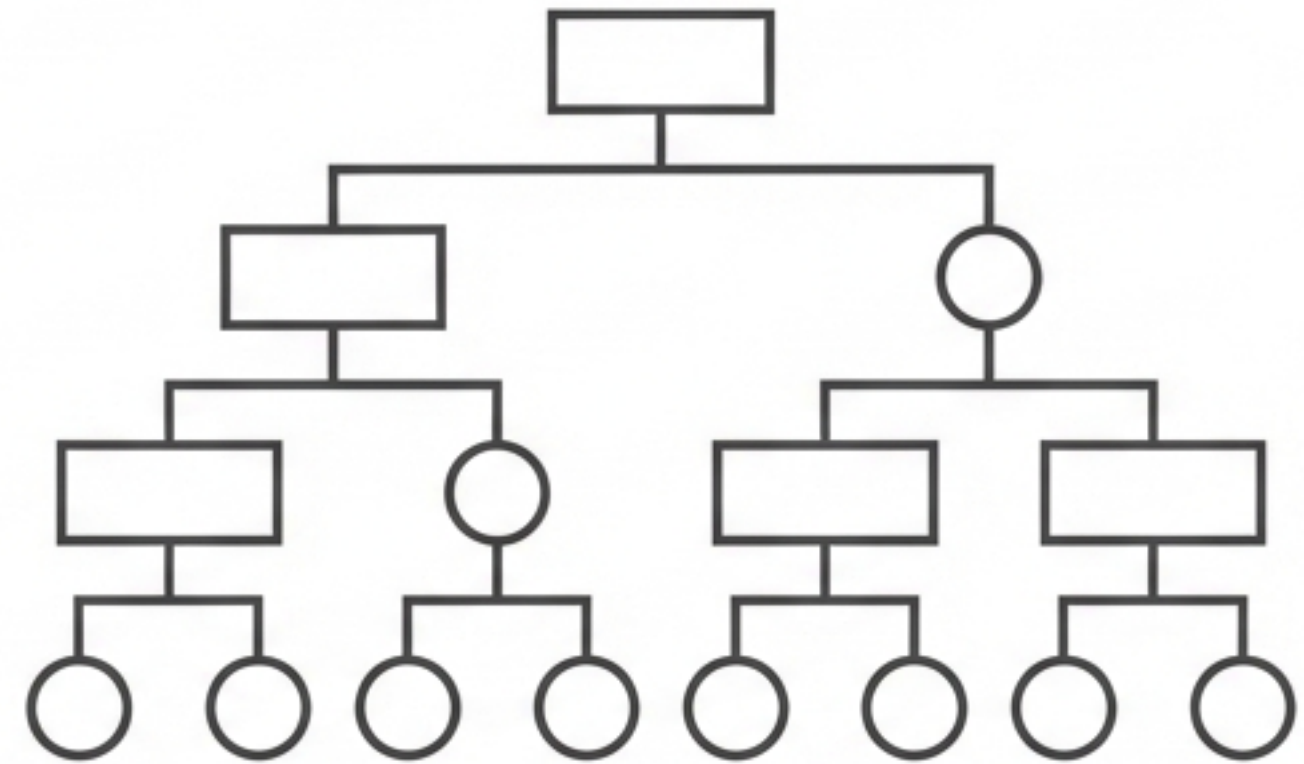
# The Gaussian Obsession: Why Normality Matters

Linear Models  
(e.g., Logistic Regression)



Assumes Normality. Performance degrades on skewed data.

Tree Models  
(e.g., Decision Trees)



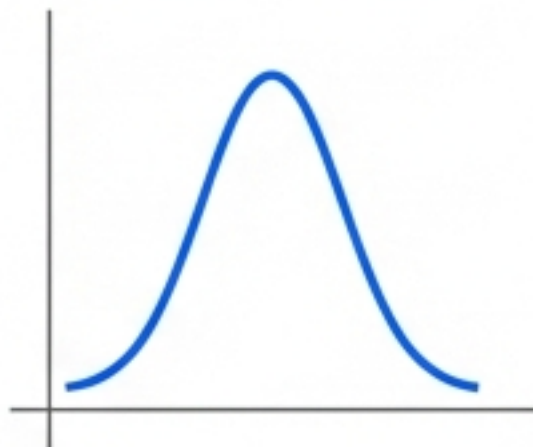
Distribution Agnostic. Splits based on value thresholds, not probability curves.

**Statistics “loves” the Normal Distribution.  
It is the happy place where math becomes simple.**

# Diagnosing the Distribution

## The Diagnostic Toolkit

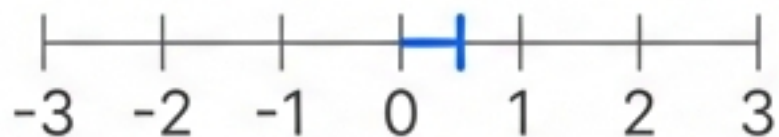
### PDF (Probability Density Function)



Visual Inspection via  
``sns.distplot``

⦿ Subjective

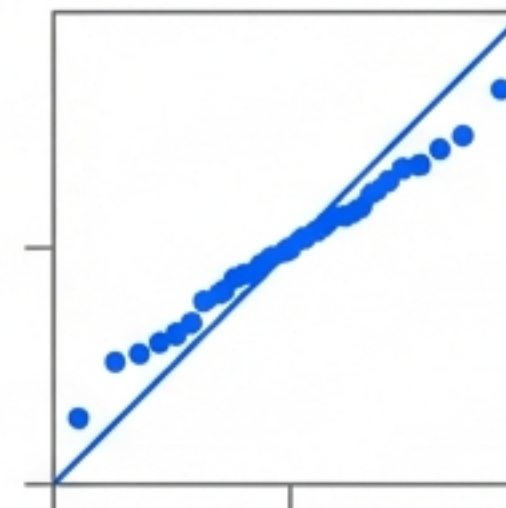
### Skewness Score



Calculation with  
``pandas.skew()``  
0 = Normal. +/- = Skewed.

⦿ Quantitative

### Q-Q Plot

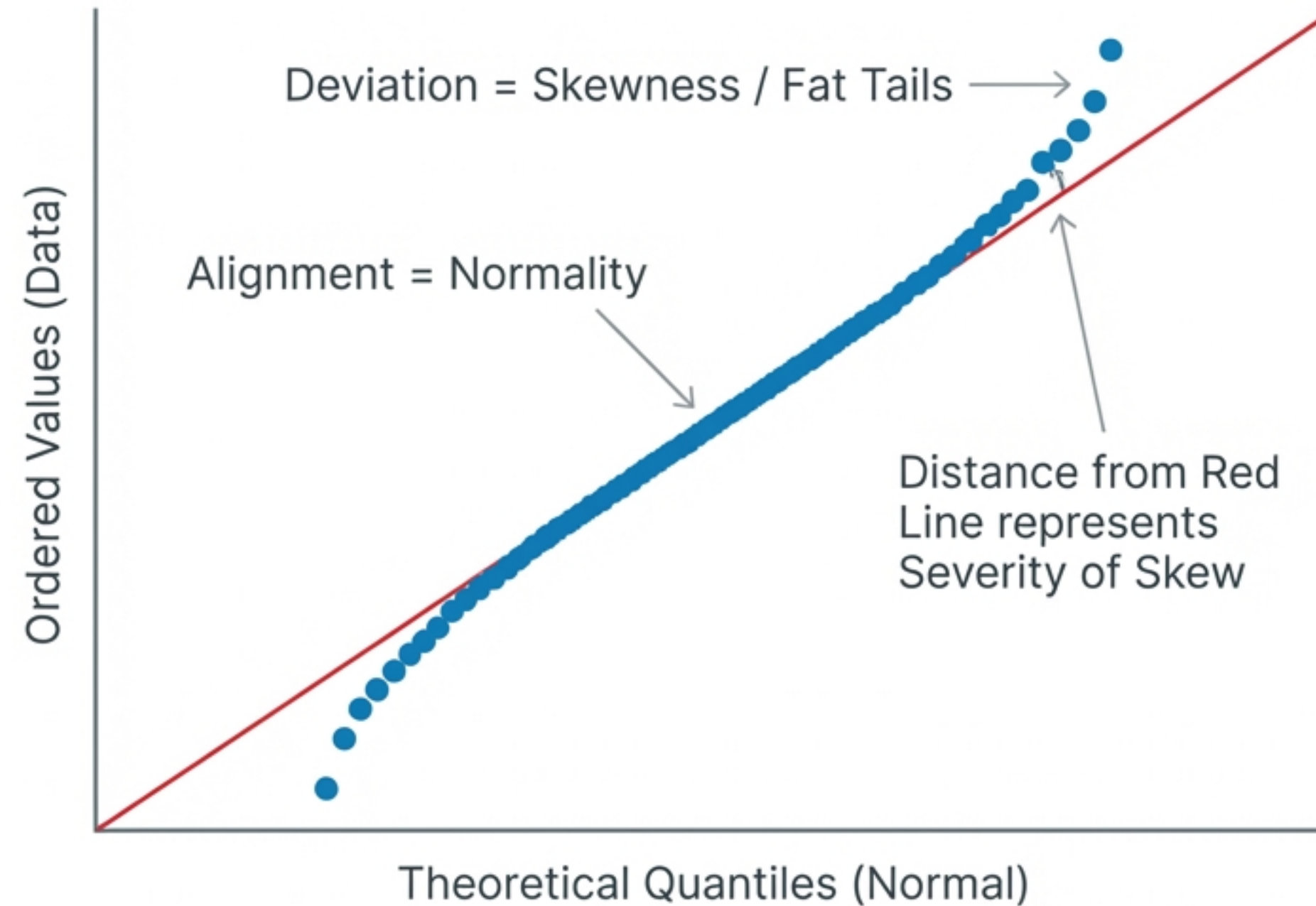


Quantile-Quantile Plot  
The Gold Standard for  
diagnosis.

⦿ Definitive



# Visual Literacy: Reading the Q-Q Plot



Rule of Thumb: If the blue dots hug the red line, the patient is healthy (Normal). If they peel away, treatment is required.

# The Treatment Tool: Scikit-Learn FunctionTransformer

```
untitled.py

from sklearn.preprocessing import FunctionTransformer
import numpy as np

# 1. Define the Transformer (e.g., Log Transform)
# handling zeros with log1p (log(1+x))
trf = FunctionTransformer(func=np.log1p)
trf = FunctionTransformer(func=np.log1p)

# 2. Apply to Data
X_transformed = trf.fit_transform(X)
```

## Flexibility & Safety

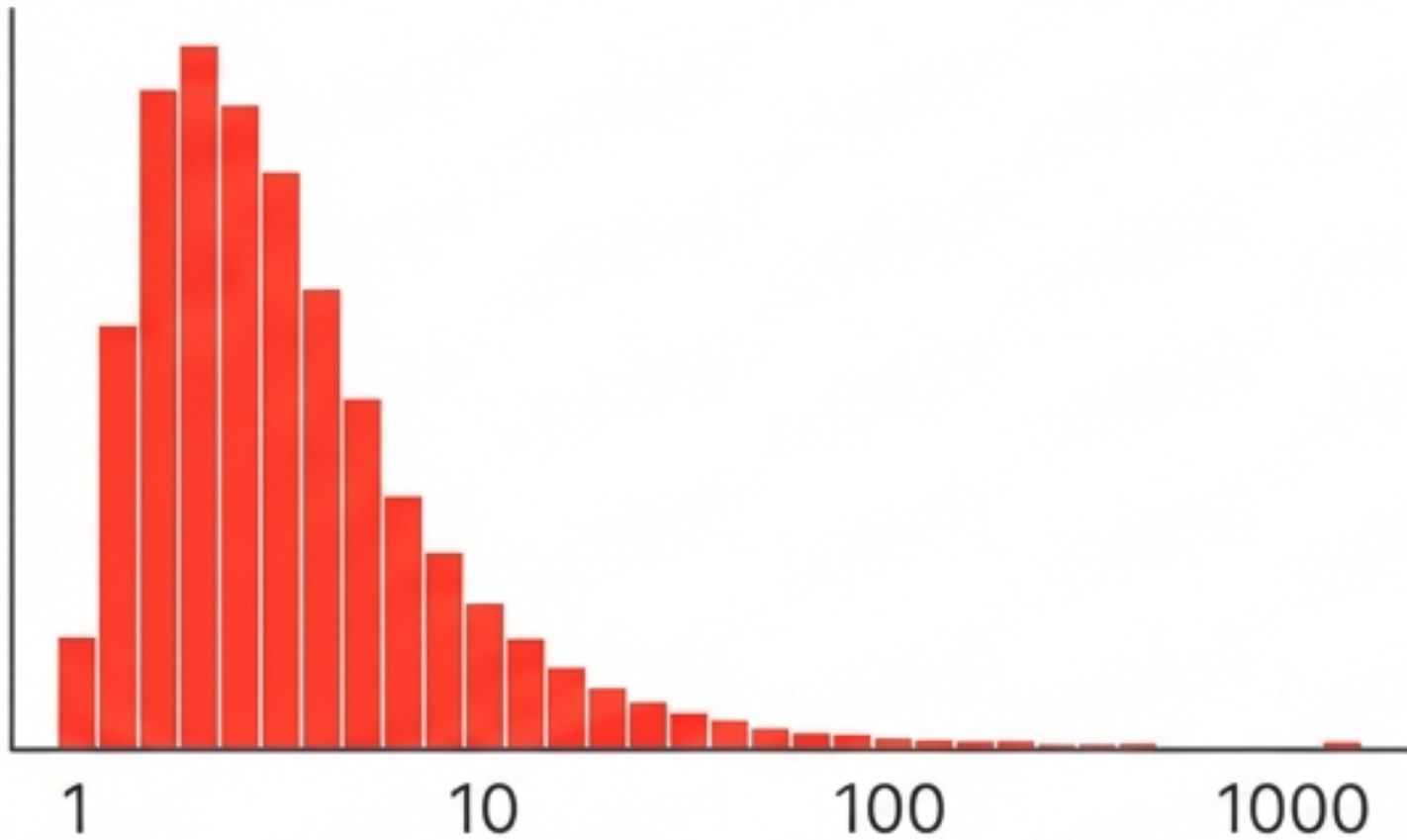
- **Flexibility:** Maps any mathematical function to a feature.
- **Safety:** Use ``np.log1p`` instead of ``np.log`` to prevent errors with zero values.



# Treatment A: The Log Transform

The Gold Standard for Right-Skewed Data

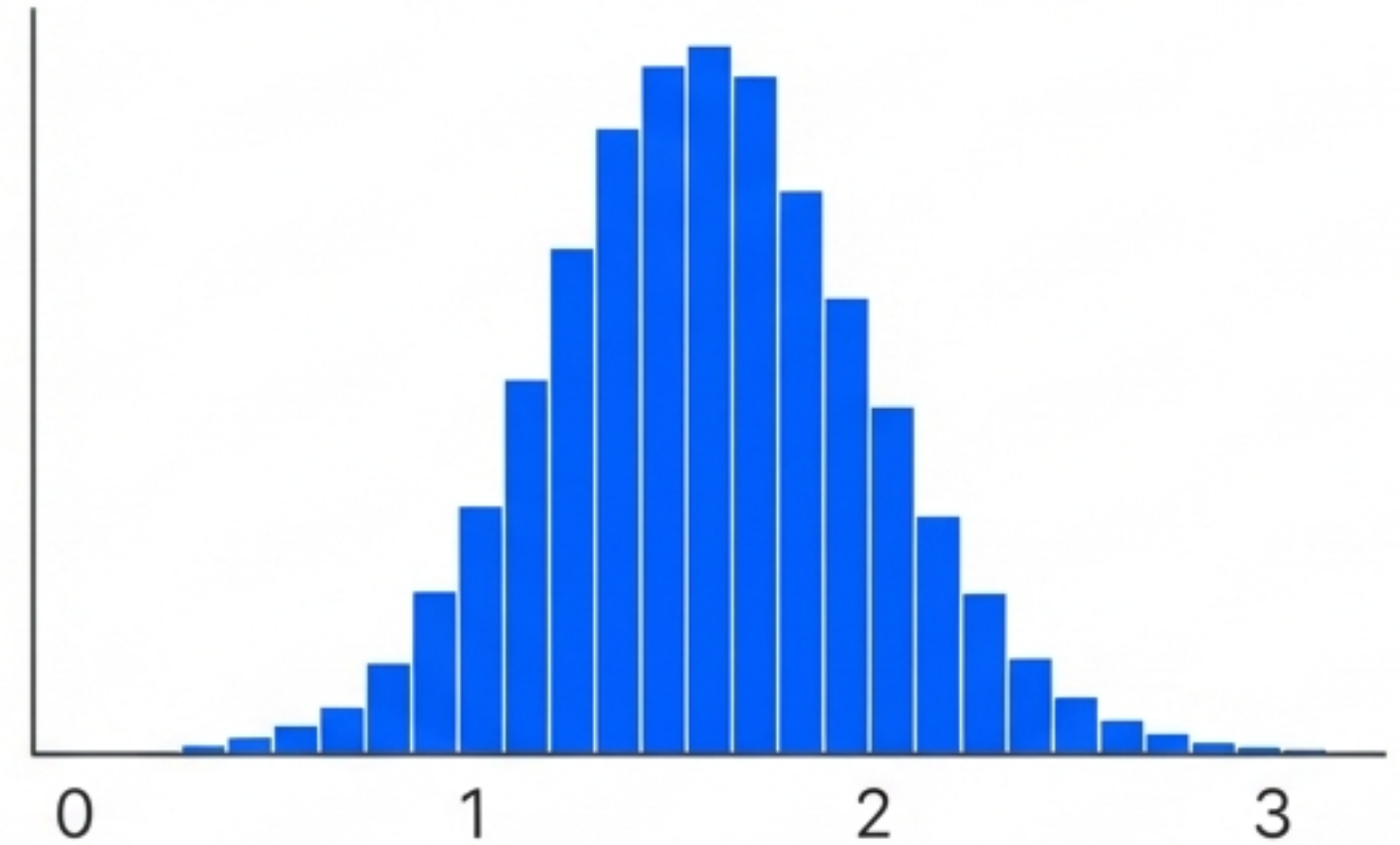
Raw Scale (Multiplicative)



$\text{Log}(x)$



Log Scale (Additive)



**Mechanism:** Compresses the scale of large values. It pulls the long tail back toward the center.

**Contraindication:** Cannot be used on negative values.

# Alternative Treatments

## Reciprocal Transform

$$1/x$$

Reverses the order. Large becomes small, small becomes large.

**Note:** Unstable with zero values. Inter

## Square Root Transform

$$\sqrt{x}$$

A milder version of Log. Good for right-skewed data where Log is too aggressive.

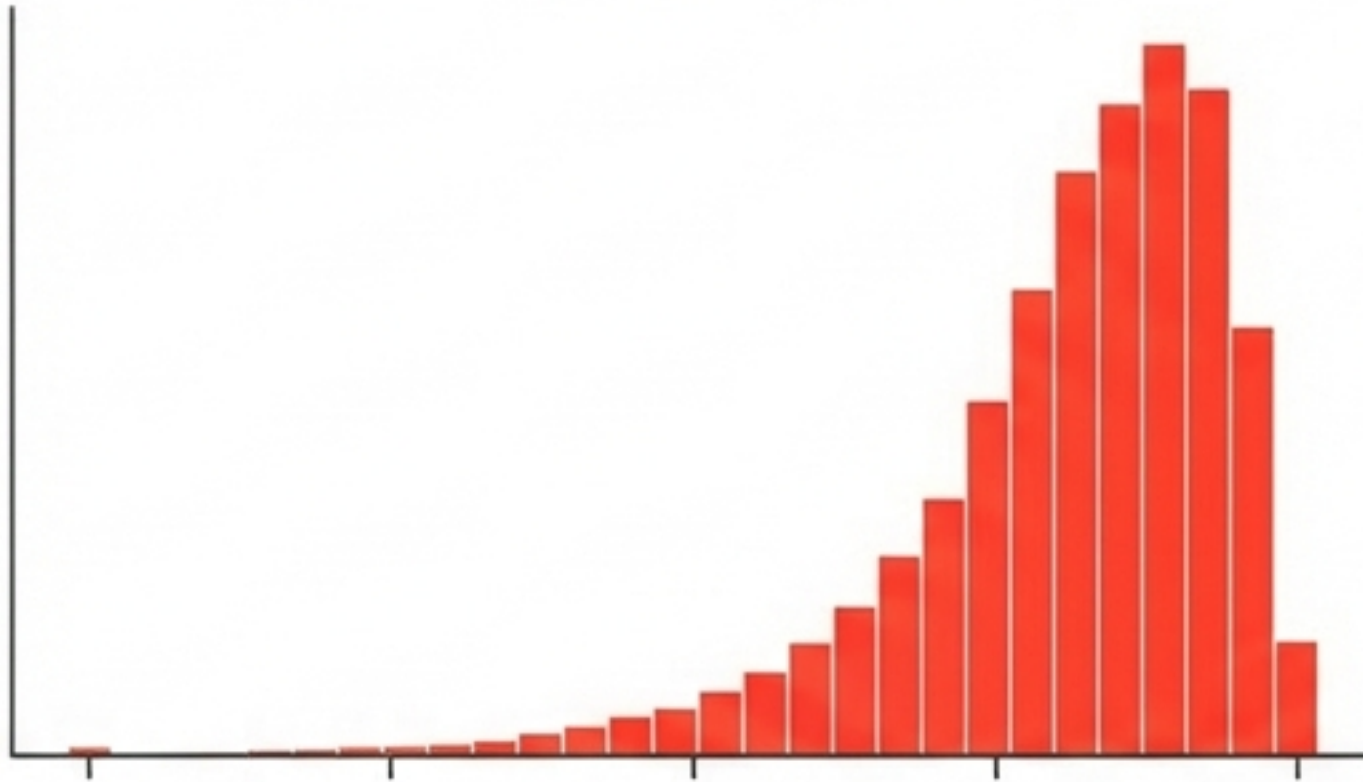




# Treatment C: The Square Transform

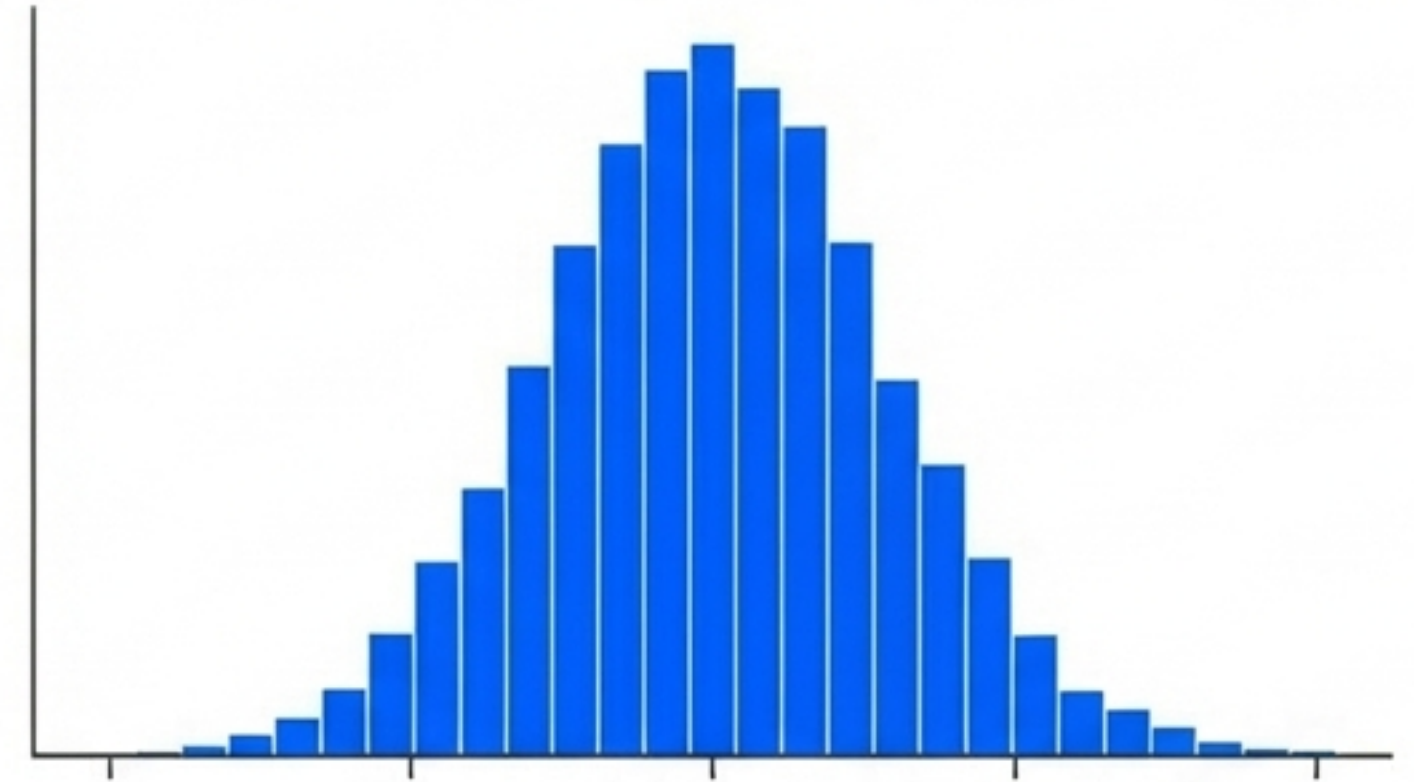
## Correcting Left-Skewed Data

Left Skewed



Power  
Transform ( $x^2$ )  
→

Bell Curve

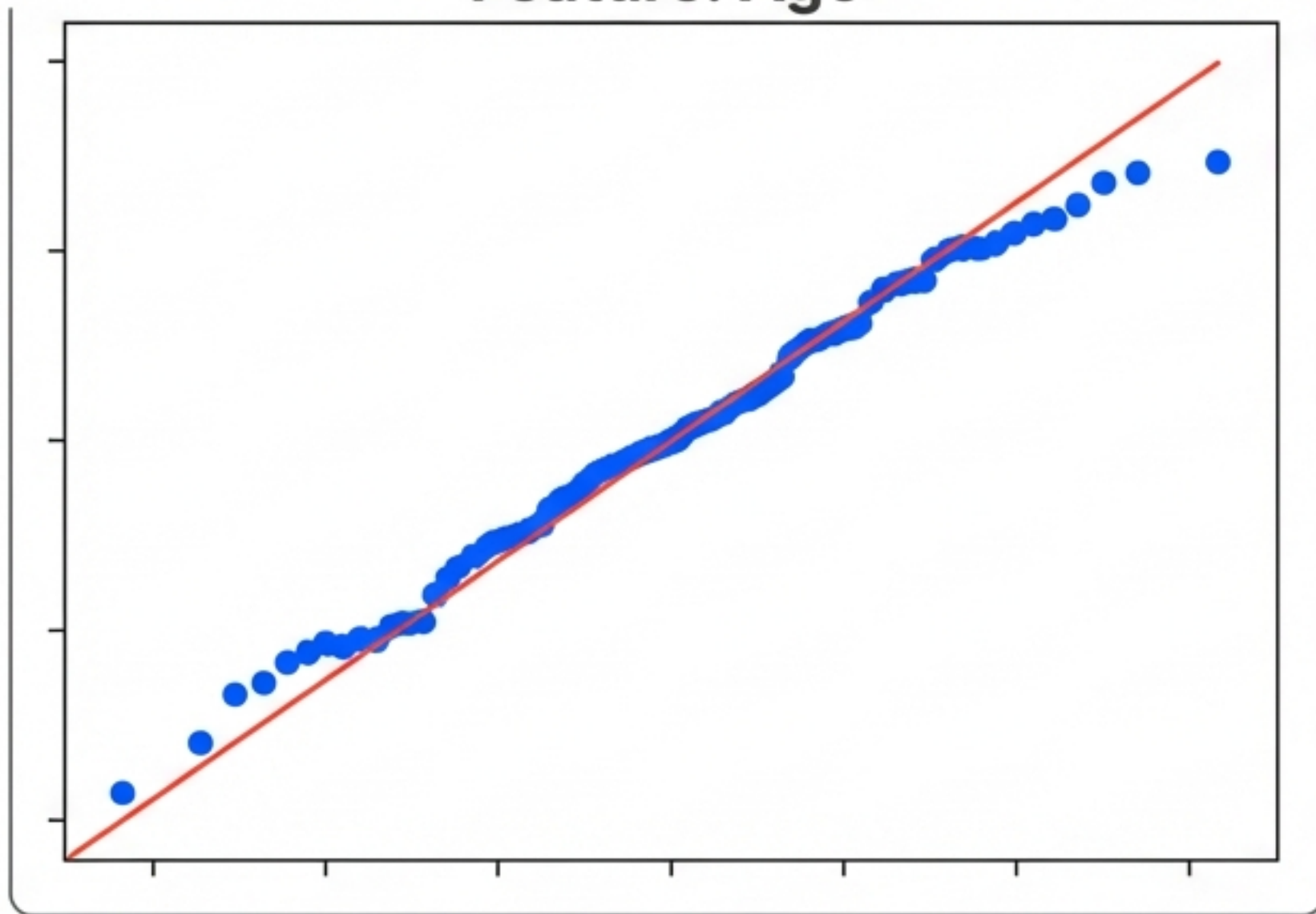


Mechanism: Expands the distance between smaller values, countering the bunching at the high end.

# Case Study: Titanic Dataset

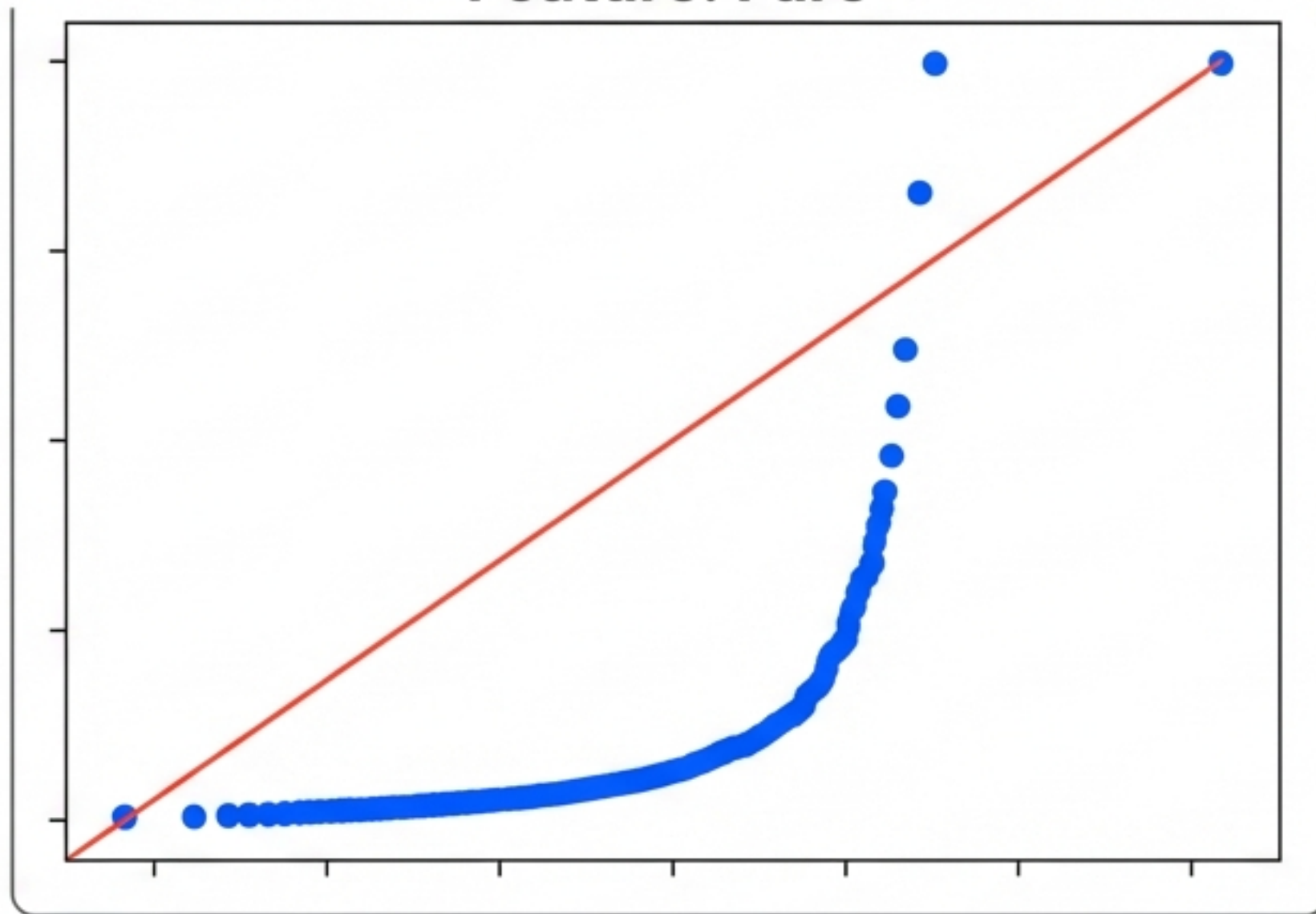
## Initial Diagnosis of Features

Feature: Age



Status: Roughly Normal

Feature: Fare



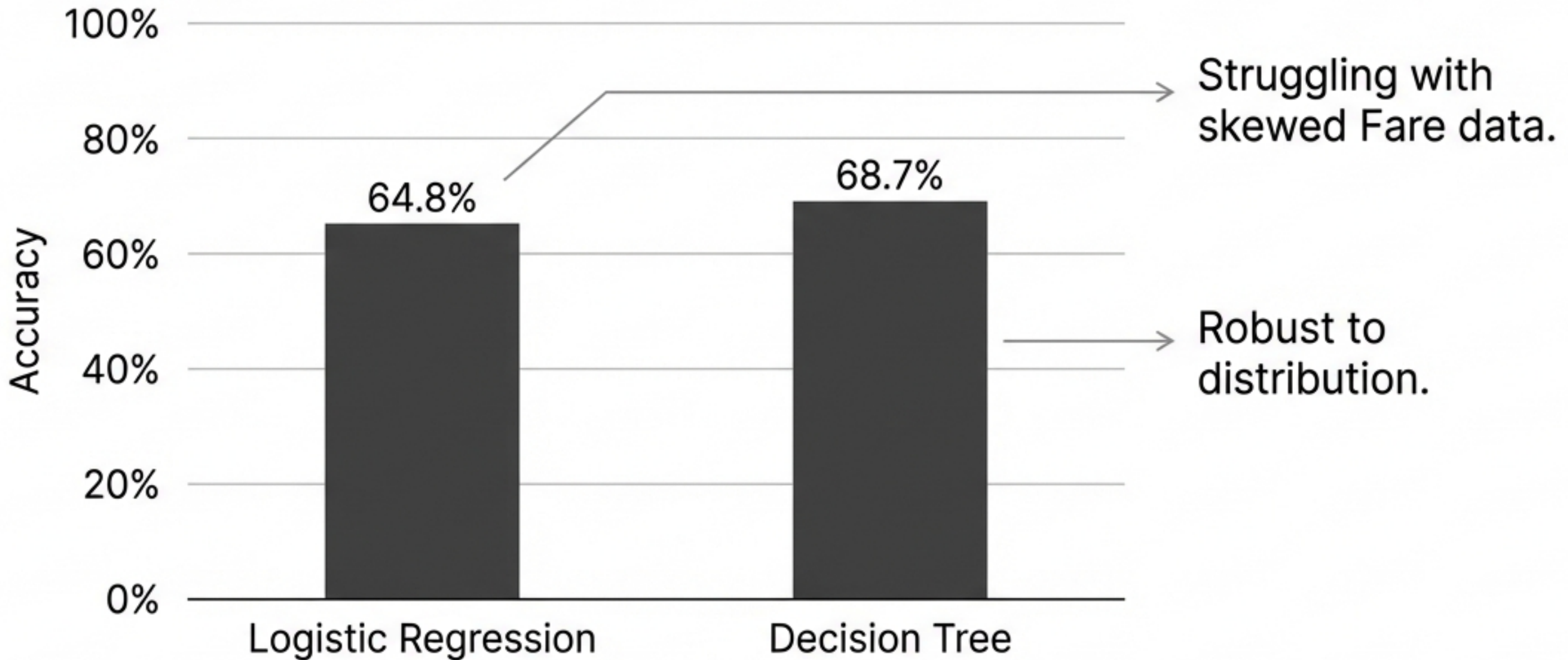
Status: Critical Right Skew (Outliers)

Diagnosis: "Age" is healthy. "Fare" exhibits severe non-normality due to extreme ticket prices.



# Experiment Phase 1: Baseline Performance

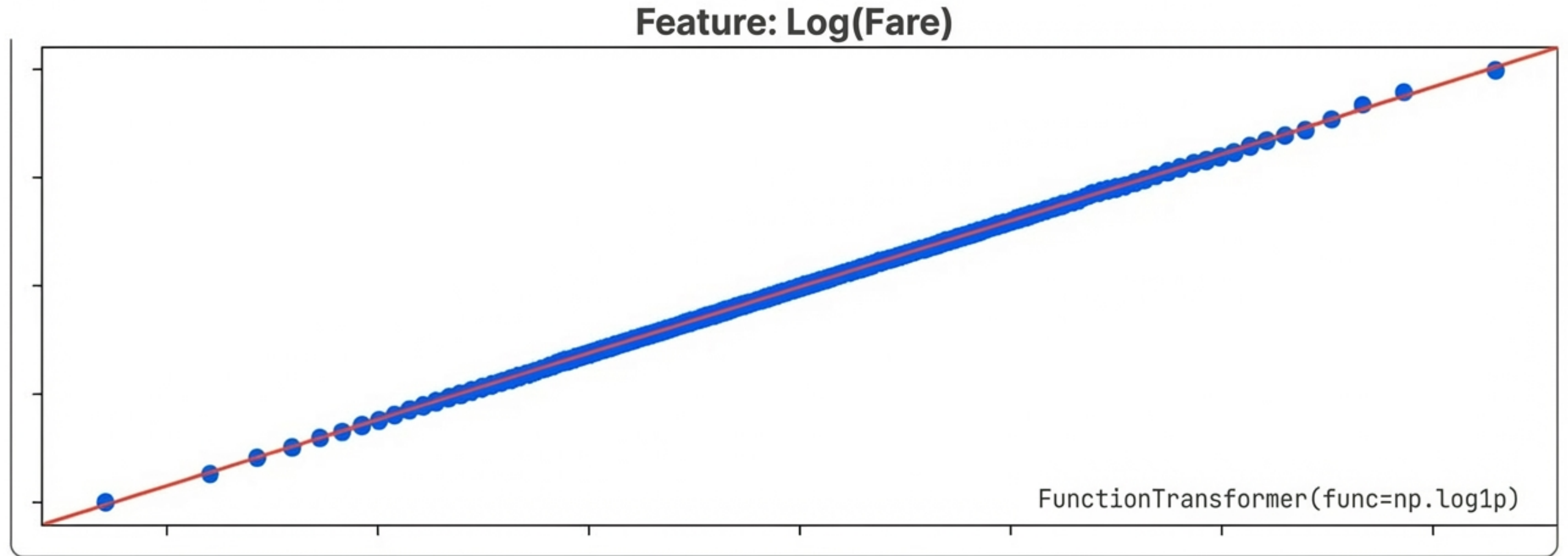
Training on Raw, Untransformed Data



Observation: The Linear model is significantly underperforming the Tree model.

# Applying the Cure

Intervention: Log Transform on 'Fare' column

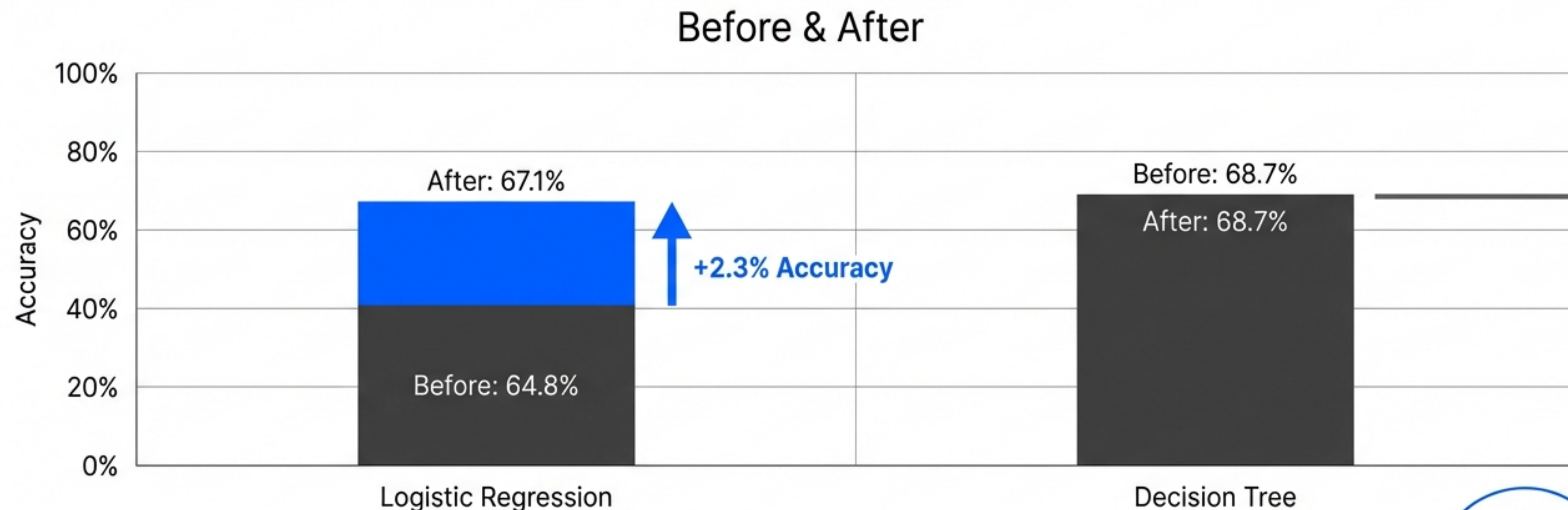


Result: The distribution has successfully mimicked Normality.



# The Post-Treatment Prognosis

## Performance Uplift Validation

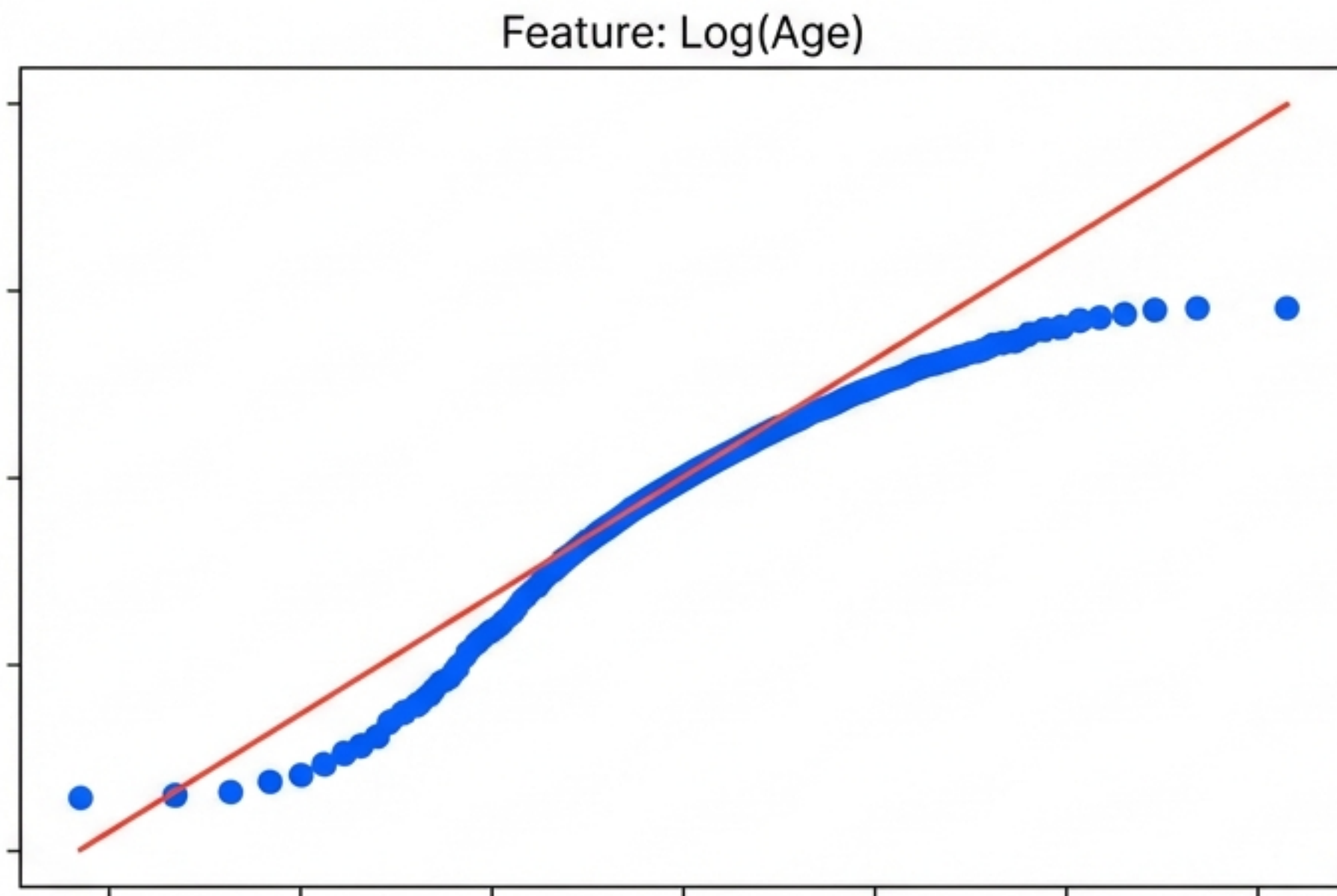


Conclusion: Mathematical transforms unlock the hidden potential of Linear Models. Tree models remain unaffected.



# Caution: Do Not Over-Prescribe

## The Risk of Treating Healthy Features



**Action:** Applied Log Transform to "Age" (Already Normal).

**Outcome:** Model Accuracy Decreased.

**Lesson:** Only treat features that are sick. Verify with a Q-Q plot post-transform.



# Practical Heuristics in Helvetica Now Display bold

## Right Skewed?

**log** **Action:** →  
Try Log Transform.

**Fallback:** If Log is too strong, try Square Root.

**Safety:** Use ``log1p`` for zeros.

## Left Skewed?

**$x^2$**  **Action:** →  
Try Square ( $x^2$ ) or Exponential.

## Validation Protocol

**✓** **Action:**  
1. Check Q-Q Plot.  
2. Check Accuracy Metric.

“If the plot looks better but accuracy drops, trust the accuracy.”

Implementation: ``sklearn.preprocessing.FunctionTransformer``